

MiWay Transit Optimizer — methods one-pager

A one-page summary for professors, municipal staff, and referees. Every number below is mechanically verified against a repository artifact (`verify_uni_docs.py`); the two frozen public artifacts are linked. Companion to the full methods appendix in the project repo.

The question

MiWay (Mississauga, pop. ~700k) publishes timetables and a real-time feed, but no agency publishes where riders actually lose time. One question drives the project: ****where does the promise of the timetable break down, by how much, and can it be fixed without new service?**** — answered with a measurement pipeline, a formal optimization model, and pre-registered falsifiable predictions scored mechanically.

Data

- **Live GTFS-RT collection.** A personal always-on collector polls the VehiclePositions + TripUpdates feeds around the clock since 2026-08-17 (cloud VPS since Sept 9); 66,000+ polls at 99.0% success, 4M+ unique departures observed.
- **Public timetable vintages,** frozen and SHA-256 hashed before and after the Sept 7, 2026 schedule changeover, so every before/after comparison is pinned to exact feed bytes.
- **Primary field data.** 399 on-board rider check-ins across 14 routes, personally collected — used to audit the vendor’s automatic passenger counters (APC).
- **Published dataset** (Zenodo, DOI 10.5281/zenodo.22820449): a seed-pinned 250k-row observed-lateness sample plus the full dated-actuals corpus.

Method

1. **Measure.** Record every observed departure against the scheduled timetable (recorded departures only — vendor forecasts are excluded because they are early by construction). Derived views: lateness distributions, early-departure shares, ghost (never-run) trips, realized transfer waits at every timed connection.
2. **Optimize.** A CP-SAT (OR-Tools) model proposes small timetable shifts (± 5 min per route) minimizing weighted transfer waits, with hard equity constraints capping how much any ward can get worse. Every result ships with per-ward constraint status and a reproducible planner package.
3. **Pre-register, then score.** Predictions of the Sept 7 changeover’s effects were committed with numeric falsification bands **before** the change, frozen with SHA-256 manifests; a mechanical scorecard grades them — misses published alongside hits.

Key findings (frozen cuts, not estimates from memory)

Finding	Number	Basis
Early departures at school stops fell after the changeover	62% → 39%	pre-registered prediction, confirmed (v4 freeze)
Ghost trips after the changeover	1-in-255 of verifiable trips	121 of 30,848 (v4 freeze)
Ghost-trip estimate, self-caught and publicly corrected	1-in-32 → 1-in-60	published correction + vintage audit
Modeled wait reduction from re-timing (simulated)	~9,700 passenger-min/weekday	CP-SAT plan; wins on 500/500 simulated days built from real observed lateness; pilot pending
APC reliability (personal audit vs vendor sensors)	72% of load changes show no display change	399 ride-along check-ins

Known limits (stated, not buried)

- The optimizer result is a **model** result on a static schedule; every quotation carries the honest validation (Monte Carlo over observed lateness, worst-case sensitivity), and a field pilot is the required next step.
- Boardings outside APC-equipped routes are calibrated estimates, kept separate from measured lower bounds.
- The ghost-rate detector changed vintage mid-window (documented); the pre- and post-change rates are not directly comparable without the audit.

Reproduce it

The public demo replays 25 frozen API snapshots from the live system — every view labels its evidence basis; its footer links the exact commit and dataset DOI behind the numbers on screen. The methods appendix documents every script and artifact hash. Demo: tap1338.github.io/miway-findings/demo · Dataset: doi.org/10.5281/zenodo.22820449 · Findings: github.com/Tap1338/miway-findings